

Java2Compiler - Test Documentation

Running Tests

The Java2Compiler project includes a comprehensive test suite managed by TestRunner.

How to Run Tests

To execute all tests, compile the project and run the following command in your terminal:

Command:

```
java tests.TestRunner
```

The test runner automatically discovers all test case files in `tests/cases/` and runs them, reporting:

- Valid tests: Should execute without throwing exceptions.
 - Invalid tests: Should throw either `LexicalException` (lexer errors) or `ParseException` (parser errors).
-

Test Cases

Test 1: Full Program (`valid_full_program.txt`)

- Type: Valid
- Description: Tests a complete program with comments, assignments, arithmetic operations, boolean expressions, and print statements.
- Test Code:

```
tests > cases > ≡ valid_full_program.txt
1  `start comment`
2  x :- 10 ~
3  y :- x + 5 * 2 ~
4  ok :- (:) : { : ! : ( : [ : ) ~
5  leviosa -> y -< ~
6  cmp :- 3 >= 2 : { 1 < 2 ~
7
```

- Expected Behavior: Program should execute without errors, performing variable assignments and printing output.
-

Test 2: Grouping and Precedence (`valid_grouping_precedence.txt`)

- Type: Valid
- Description: Tests operator precedence and grouping with parentheses.
- Test Code

```
tests > cases > ≡ valid_grouping_precedence.txt
1  total :- -> 1 + 2 -< * 3 ~
2  leviosa -> total -< ~
3  █
```

- Expected Behavior: Should correctly evaluate $(1 + 2) * 3 = 9$ and print the result.
-

Test 3: Chained Comparison (`invalid_chained_comparison.txt`)

- Type: Invalid
- Description: Tests that chained comparisons are rejected (e.g., $1 < 2 < 3$).
- Test Code:

```
tests > cases > ≡ invalid_chained_comparison.txt
1  `Test chained comparison error`
2  a :- 5 ~
3  b :- 10 ~
4  `This should fail because comparisons cannot chain`
5  result :- 1 < 2 < 3 ~
6  leviosa -> result -< ~
7
```

- Expected Behavior: `ParseException` should be thrown - comparisons cannot chain.
-

Test 4: Expect Expression (`invalid_expect_expression.txt`)

- Type: Invalid
- Description: Tests error handling when an expression is expected but not provided.
- Test Code:

```
tests > cases > ≡ invalid_expect_expression.txt
1  `Test missing expression in assignment`
2  a :- 5 ~
3  b :- 10 ~
4  `Missing expression after assignment operator`
5  c :- ~
6
```

- Expected Behavior: `ParseException` should be thrown.
-

Test 5: Expect Statement (`invalid_expect_statement.txt`)

- Type: Invalid
- Description: Tests error handling when a statement is expected but not provided.
- Test Code:

```
tests > cases > ≡ invalid_expect_statement.txt
1  `Test statement with no valid content`
2  x :- 42 ~
3  leviosa -> x -<~
4  `Invalid empty statement`
5  ~
```

- Expected Behavior: `ParseException` should be thrown.
-

Test 6: Identifier with Digits (`invalid_identifier_digits.txt`)

- Type: Invalid
- Description: Tests rejection of invalid identifiers containing digits.

- Test Code:

```
tests > cases > ≡ invalid_identifier_digits.txt
1  `Test identifier with digits error`
2  x :- 100 ~
3  `Identifiers cannot contain digits`
4  abc1 :- 50 ~
5  leviosa -> abc1 ~
6
```

- Expected Behavior: `LexicalException` should be thrown - identifiers cannot contain digits.
-

Test 7: Missing Assignment Operator (`invalid_missing_assign.txt`)

- Type: Invalid
- Description: Tests error when assignment operator (`:-`) is missing.
- Test Code:

```
tests > cases > ≡ invalid_missing_assign.txt
1  `Test missing assignment operator`
2  a :- 10 ~
3  b :- 20 ~
4  `Missing :- operator`
5  c 30 ~
6  leviosa -> a -<~
7
```

- Expected Behavior: `ParseException` should be thrown.
-

Test 8: Missing Statement Terminator on Assignment

(`invalid_missing_end_assignment.txt`)

- Type: Invalid
- Description: Tests error when statement terminator (`~`) is missing from an assignment.

- Test Code:

```
tests > cases > ≡ invalid_missing_end_assignment.txt
1  `Test missing statement terminator`
2  a :- 15 ~
3  b :- 25 ~
4  `Missing ~ terminator on this assignment`
5  c :- 35
6  leviosa -> c -<~
7
```

- Expected Behavior: `ParseException` should be thrown.
-

Test 9: Missing Statement Terminator on Print

(invalid_missing_end_print.txt)

- Type: Invalid
- Description: Tests error when statement terminator (~) is missing from a print statement.
- Test Code:

```
tests > cases > ≡ invalid_missing_end_print.txt
1  `Test missing terminator on print`
2  x :- 99 ~
3  y :- 88 ~
4  `Missing ~ on this print statement`
5  leviosa -> y -<
6  z :- 77 ~
7
```

- Expected Behavior: `ParseException` should be thrown.
-

Test 10: Missing Right Parenthesis (invalid_missing_rparen.txt)

- Type: Invalid
- Description: Tests error when closing parenthesis (-<) is missing from a grouped expression.

- ```
tests > cases > invalid_missing_rparen.txt
1 `Test missing closing parenthesis`
2 x :- 5 ~
3 y :- 10 ~
4 `Missing closing -< parenthesis`
5 z :- -> x + y ~
6 leviosa -> z -< ~
7
```

- ## Test 11: Number Overflow (`invalid_number_overflow.txt`)

- ```
tests > cases > invalid_number_overflow.txt
1  `Test number overflow error`
2  a :- 100 ~
3  b :- 200 ~
4  `Number too large to fit in integer`
5  c :- 99999999999999999999 ~
6  leviosa -> c -<~
7
```

- ## Test 12: Unexpected Bang (`invalid_unexpected_bang.txt`)

- ```
tests > cases > invalid_unexpected_bang.txt
1 `Test unexpected bang character`
2 x :- 42 ~
3 `Invalid ! outside of NOT operator context`
4 !
5 leviosa -> x -< ~
6
```

- Expected Behavior: `LexicalException` should be thrown.

---

### Test 13: Unexpected Character (`invalid_unexpected_char.txt`)

- Type: Invalid
- Description: Tests rejection of invalid characters (e.g., @).
- Test Code:

```
tests > cases > ≡ invalid_unexpected_char.txt
1 `invalid character`
2 leviosa -> @ -<~
3 |
```

- Expected Behavior: `LexicalException` should be thrown - unexpected character.

---

### Test 14: Unexpected Colon (`invalid_unexpected_colon.txt`)

- Type: Invalid
- Description: Tests error when `:` appears outside of valid operator contexts.
- Test Code:

```
tests > cases > ≡ invalid_unexpected_colon.txt
1 `Test unexpected colon character`
2 x :- 77 ~
3 y :- 88 ~
4 `Invalid : outside operator context`
5 :
6 leviosa -> x -<~
```

- Expected Behavior: `LexicalException` should be thrown.

---

### Test 15: Unterminated Comment (`invalid_unterminated_comment.txt`)

- Type: Invalid
- Description: Tests rejection of comments that are not properly closed.

- Test Code:

```
tests > cases > ≡ invalid_terminated_comment.txt
1 `Test unterminated comment error`
2 x :- 55 ~
3 y :- 66 ~
4 `Unterminated comment with no closing backtick`
5 z :- `this comment is never closed
6 leviosa -> z -<~
```

- Expected Behavior: `LexicalException` should be thrown - unterminated comment.

---

## Test Execution Summary

| Test # | Test Name               | Type    | Expected Result      |
|--------|-------------------------|---------|----------------------|
| 1      | Full Program            | Valid   | Execute successfully |
| 2      | Grouping and Precedence | Valid   | Execute successfully |
| 3      | Chained Comparison      | Invalid | ParseException       |
| 4      | Expect Expression       | Invalid | ParseException       |
| 5      | Expect Statement        | Invalid | ParseException       |
| 6      | Identifier with Digits  | Invalid | LexicalException     |
| 7      | Missing Assignment      | Invalid | ParseException       |
| 8      | Missing End Assignment  | Invalid | ParseException       |
| 9      | Missing End Print       | Invalid | ParseException       |
| 10     | Missing Right Paren     | Invalid | ParseException       |



|    |                         |         |                  |
|----|-------------------------|---------|------------------|
| 11 | Number Overflow         | Invalid | LexicalException |
| 12 | Unexpected Bang         | Invalid | LexicalException |
| 13 | Unexpected Char         | Invalid | LexicalException |
| 14 | Unexpected Colon        | Invalid | LexicalException |
| 15 | Unterminated<br>Comment | Invalid | LexicalException |